

Ce que le cours a enseigné, et ce qui en a été fait

Reconstitution des leçons de la vidéo de formation (**Claude Code Masterclass**) réellement collées dans le chat par Ilies entre le 4 et le 12 août 2026, organisée selon la vraie table des matières du cours — pas une timeline par date.

Méthode

8 sessions Claude Code ont été analysées (`bef46bcb` , `bc2538f9` , `a056c64e` , `8d281e45` , `3bb6cf8f` , `6b055b65` , `62cf38a7` , `ecc9fb7c`). Toutes les images collées dans le chat ont été extraites, regardées une par une, triées entre captures réelles de la vidéo du formateur et captures personnelles de debug (Firebase, terminal, Figma) — écartées de ces fichiers. Chaque leçon retenue est reliée au commit git exact qui l'a appliquée dans le code de Pocket Heist.

Sommaire

Section	Contenu retrouvé
Section 1 — Introduction	Aucun — les captures commencent après cette section
Section 2 — Commands, Context, Tools & Hooks	Hooks <code>PostToolUse</code> , <code>log jq</code> , chaînage avec <code>prettier</code> , commandes <code>/commit-message</code> et <code>/component</code>
Section 3 — Plan Mode & Specs	Template de spec, méthode de rédaction complète, prompt <code>/spec</code> , du spec au plan d'implémentation

Section	Contenu retrouvé
Section 4 — MCP Servers	Plan Firebase MCP (Firestore + Auth) — Figma MCP et Context7 utilisés mais non captés depuis la vidéo
Section 5 — Plugins & Skills	Aucun — rien de capté ne correspond à ce thème

À lire avec ces nuances

- Deux sections (1 et 5) n'ont aucune leçon confirmée : ce n'est pas un oubli, c'est qu'aucune capture des 8 sessions ne s'y rattache réellement. Mieux vaut le dire que forcer un contenu.
- Certaines leçons chevauchent plusieurs sections (ex. la commande `/spec` est une «Command » mais son contenu pédagogique enseigne la méthode de «Specs ») — le chevauchement est signalé dans le fichier concerné plutôt que dupliqué.
- Chaque fichier cite les commits git exacts (hash court, message, date) qui ont appliqué la leçon dans le projet — vérifiables avec `git show <hash>`.

Section 1 — Introduction

Cours : Claude Code Masterclass — Section 1 (8/8 leçons, 39 min).

Après lecture des 8 rapports de session (`bef46bcb` , `bc2538f9` , `a056c64e` , `8d281e45` , `3bb6cf8f` , `6b055b65` , `62cf38a7` , `ecc9fb7c`), **aucune capture collée par Ilies ne correspond à la Section 1 — Introduction** du cours Udemy.

Pourquoi rien n'a été rattaché ici

Toutes les captures identifiées dans les 8 sessions couvrent des sujets qui relèvent clairement des sections suivantes :

- **Section 2 (Commands, Context, Tools & Hooks)** : configuration du hook `PostToolUse` via `/hooks` (sessions `bc2538f9` et `a056c64e`).
- **Section 3 (Plan Mode & Specs)** : template de spec, specs complètes du formateur (login/signup, `useUser` , logout, login form, route protection, create heist form) dans quasiment toutes les sessions à partir du 6 août.
- **Section 4 (MCP Servers)** : Firebase MCP (plan Firestore + Auth) dans `8d281e45` et `3bb6cf8f` .
- **Section 5 (Plugins & Skills)** : rien de confirmé non plus (voir ce fichier dédié).

Un seul indice indirect existe : dans la session `bef46bcb` (04/08), le rapport mentionne qu'Ilies avait fait un `/init` juste avant de coller ses images — mais **aucune capture n'illustre cette étape elle-même** ; les 3 images de cette session montrent en réalité toutes le même mock de «skeleton loader», sans rapport avec une introduction au cours ou à l'outil.

De même, l'image de la session `8d281e45` qui montre l'interface Udemy (barre de progression du cours) affiche «Section 2 : 13/13 », «Section 3 : 6/6 », «Section 4 : 6/13 » — **pas la Section 1** — et sert uniquement de repère chronologique pour un récapitulatif demandé par Ilies, pas d'un contenu pédagogique de la Section 1.

Conclusion

Il n'y a pas de leçon à documenter pour la Section 1 — Introduction. Les 8 sessions capturées par Ilies commencent après cette section : la première capture exploitable démarre directement dans le vif du sujet (hooks, Section 2). Aucun commit du projet n'est donc à rattacher ici.

Section 2 — Commands, Context, Tools & Hooks

Cours : Claude Code Masterclass — Section 2 (13/13 leçons, 1h33).

Cette section du cours couvre les hooks Claude Code (via `/hooks`) et les commandes personnalisées du projet. Deux sessions collent précisément à ce thème : **bc2538f9** (05/08) et **a056c64e** (06/08), toutes deux confirmées par un watermark «Udemy» ou un chemin `/Users/shaun/...` (machine du formateur) visible sur les captures.

Créer un hook `PostToolUse` pas à pas via `/hooks`

Sessions source : bc2538f9 (05/08) et a056c64e (06/08)

Ce que montre la vidéo : l'écran interactif `/hooks` en 4 étapes — choix de l'événement `PostToolUse`, puis `Add new matcher...` avec la liste des `tool_name` valides (`Task`, `Bash`, `Read`, `Edit`, `Write`, ...) et la syntaxe (`Write` seul, `Write|Edit` pour plusieurs outils, `Web.*` en regex), puis retour à `/hooks` titré «Matcher: Write|Edit», puis `Add new hook...` avec le champ `Command:` rempli avec `echo "hello"`.

Ce que le formateur explique : un hook se configure en deux temps — d'abord un **matcher** (quel outil déclenche le hook), puis une **commande shell** exécutée après. L'entrée reçue par la commande est un objet JSON (`tool_input` / `tool_response`) ; le code de sortie détermine l'affichage (0 = stdout visible directement dans le transcript, sans besoin de `Ctrl+O` — point confirmé par une capture vidéo montrant le résultat `PostToolUse:Edit hook succeeded: hello` juste sous un diff).

Application dans Pocket Heist : le menu interactif `/hooks` ne s'est pas comporté comme attendu dans l'environnement WSL d'Illies ; la solution a été d'éditer directement `.claude/settings.local.json` avec l'équivalent JSON (`"matcher": "Write|Edit"`, `"command": "echo \"hello\""`). Cette compréhension a ensuite été rédigée en documentation pédagogique dans `.claude/lecons/hooks-guide.md` et `.claude/lecons/hooks-plan.md` — commit `19bfc5a` («docs: ajoute les leçons hooks et traduit les commandes en français», 06/08/2026).

Logger l'objet complet transmis au hook avec `jq`

Session source : a056c64e (06/08)

Ce que montre la vidéo : le contenu d'un fichier `tool-use.json` produit par un hook, contenant l'objet complet reçu (`session_id`, `cwd`, `hook_event_name`, `tool_name`, `tool_input`, `tool_response` avec `structuredPatch`) sur un fichier du projet du formateur (`app/(public)/login/page.tsx`, chemin `/Users/shaun/Code/Courses/pocket_heist`).

Ce que le formateur explique : pour logger l'objet JSON complet reçu par le hook, le bon filtre `jq` est l'identité `jq '.'` (garde tout), et non un filtre restrictif comme `.tool_input.file_path` qui ne garderait qu'un seul champ.

Application dans Pocket Heist : correction de la commande dans `.claude/settings.local.json` (`jq . > tool-use.json`), mise à jour de `.claude/lecons/hooks-jq-log.md`, suppression d'un fichier obsolète (`tool-log.json`) — regroupé dans le commit `19bfc5a` (même commit que ci-dessus, qui ajoute justement ce fichier `hooks-jq-log.md`).

Châîner plusieurs commandes sur un même hook (permissions + auto-formatage `prettier`)

Session source : `a056c64e` (06/08)

Ce que montre la vidéo : le fichier `.claude/settings.local.json` du formateur avec des permissions pré-accordées (`WebSearch`, `Bash(git init)`, `Bash(git switch:*)`) et **trois commandes chaînées** sur le même matcher `Write|Edit` : `echo "hello"`, `jq . > tool-use.json`, et une commande de reformatage automatique des fichiers `.tsx` via `prettier`.

Ce que le formateur explique : un même matcher peut déclencher plusieurs hooks à la suite — ici un exemple concret d'auto-formatage post-édition, qui filtre sur l'extension `.tsx` avant d'appeler `npx prettier --write`.

Application dans Pocket Heist : reproduction fidèle de ces commandes dans `.claude/settings.local.json`, plus l'ajout d'un fichier `.prettierrc` (`semi: false`) pour que le hook `prettier` respecte la convention du projet (pas de points-virgules) — commit `19bfc5a` (06/08/2026).

Commande personnalisée `/commit-message`

Origine : fournie dans le squelette de départ du cours (`.claude/commands/commit-message.md`), commit `10dd3cf` («feat: ajoute le squelette de départ du projet Pocket Heist», 06/08/2026), puis traduite en français dans le commit `19bfc5a` (même date).

Ce qu'elle fait : lance `git status` et `git diff --staged`, analyse les changements stagés, puis propose un message de commit au format `<emoji> <type>: <description>` (liste fixe d'emojis : `feat`, `fix`, `refactor`, `docs`, `style`, `test`, `perf`) — sans jamais commiter automatiquement, seulement sur confirmation explicite de l'utilisateur.

Remarque : aucune capture vidéo retrouvée dans les 8 sessions ne montre spécifiquement l'écran de création de cette commande ; sa présence est déduite du diff git et du récapitulatif de la session 8d281e45, qui la cite comme exemple d'usage classé par Ilies dans «Section 2 » du cours.

Commande personnalisée `/component` (TDD)

Origine : ajoutée en français dans le commit `19bfc5a` (06/08/2026).

Ce qu'elle fait : à partir d'une description libre (`$ARGUMENTS`), détermine un nom de composant en PascalCase, écrit d'abord les tests (`tests/components/[Nom].test.tsx`), les lance (échec attendu), crée le composant (`.tsx` + `.module.css` + `index.ts`) en suivant la convention du projet, relance les tests (succès attendu), puis l'ajoute à `app/(public)/preview/page.tsx` — un cycle TDD complet piloté par une seule commande slash.

Remarque : même limite que `/commit-message` — pas de capture vidéo isolée retrouvée pour cette commande précisément ; elle est citée dans le récapitulatif de session 8d281e45 comme faisant partie du travail classé en Section 2.

Chevauchement signalé : la commande `/spec`

La commande `/spec` (créée en `d4c3bc8`, «feat: ajoute la commande /spec et son template de spécification », 06/08/2026 17:00) pourrait sembler relever de cette Section 2 («Commands ») puisque c'est une commande personnalisée comme `/commit-message` ou `/component`. **Choix fait ici : elle est classée en Section 3 «Plan Mode & Specs** », parce que tout le contenu pédagogique retrouvé à son sujet (structure du template, contenu détaillé d'une spec complète, formulation du prompt d'entrée) enseigne la **méthode de rédaction d'une spec**, pas la mécanique de création d'une commande slash — ce qui correspond exactement à l'intitulé de la Section 3, pas de la Section 2. Voir `section-3-plan-mode-specs.md`.

Note de couverture : sur les 8 sessions analysées, seules `bc2538f9` et `a056c64e` contiennent des leçons vidéo directement liées aux hooks. Aucune session ne montre d'autres sujets typiques de cette section (contexte, autres outils du terminal) — soit ce contenu n'a pas été capturé par Ilies, soit il correspond aux leçons de la Section 2 non couvertes par une capture d'écran collée dans le chat.

Section 3 — Plan Mode & Specs

Cours : Claude Code Masterclass — Section 3 (6/6 leçons, 39 min).

Cette section documente comment le formateur enseigne la méthode de rédaction de spec (via la commande `/spec` et un template fixe) puis comment il détaille cette spec en plan d'implémentation avant tout code. Les captures clés viennent des sessions **a056c64e** (structure canonique du template + exemple complet) et **3bb6cf8f** (comment formuler le prompt qui lance `/spec`, puis deux exemples complets de spec/plan).

Le template de spec canonique

Session source : a056c64e, 06/08/2026

Ce que montre la vidéo : le contenu exact de `_specs/template.md` du formateur — en-tête `# Spec for <feature-name>`, `branch:`, `figma_component (if used):`, puis les sections `## Summary`, `## Functional Requirements`, `## Figma Design Reference (only if referenced)`, `## Possible Edge Cases`, `## Acceptance Criteria`, `## Open Questions`, `## Testing Guidelines`.

Ce que le formateur explique : une spec suit toujours la même structure fixe, plus simple que ce que Claude avait improvisé au départ ; la section Figma est explicitement conditionnelle («only if referenced»), donc à omettre si aucun design Figma n'est utilisé.

Application dans Pocket Heist : réécriture de `_specs/template.md` pour coller exactement à cette structure — commit `d4c3bc8` («feat: ajoute la commande `/spec` et son template de spécification», 06/08/2026). *Chevauchement à noter : la commande `/spec` elle-même (créée dans ce même commit) relève plutôt de la [Section 2](#) «Commands» — ici on ne retient que le template de sortie qu'elle produit.*

Une spec complète et exploitable — exemple login/signup

Session source : a056c64e, 06/08/2026

Ce que montre la vidéo : la spec entièrement remplie `authentication-forms.md` du formateur pour les formulaires de connexion/inscription — un Summary concret, 9 Functional Requirements précis (validation HTML5, accessibilité ARIA, responsive...), 8 Edge Cases, 8 Acceptance Criteria, et surtout 5 Open Questions déjà tranchées par le formateur lui-même (pas de longueur minimale de mot de passe, pas de persistance, pas de «remember me», pas de «mot de passe oublié»).

Ce que le formateur explique : à quoi ressemble une spec réellement exploitable et non un simple squelette — trancher les questions ouvertes soi-même dans le document évite les allers-retours pendant le plan/l'implémentation.

Application dans Pocket Heist : réécriture complète de `_specs/login-signup-forms.md` pour intégrer les manques repérés (accessibilité, responsive, touche Entrée, absence de section Figma car non utilisée) — commit `df83e8d` («feat: ajoute les formulaires d'authentification login/signup », 10/08/2026, qui embarque spec + plan + code dans le même commit).

Formuler le prompt qui lance `/spec`

Session source : 3bb6cf8f, 10/08/2026

Ce que montre la vidéo : le texte exact tapé par le formateur pour démarrer une spec : *«/spec let's spec an auth state management solution for the app, where we can access the current user (null if logged out, the user object if logged in) — through a hook called useUser. [...] Do not spec any signup/login/logout flow yet, just a realtime global listener to update user status. »*

Ce que le formateur explique : bien formuler l'idée de fonctionnalité avant `/spec` en nommant explicitement le résultat attendu (un hook `useUser`, son comportement) et en posant une limite claire («pas de flux login/logout ») pour empêcher la spec de dériver sur un périmètre trop large.

Application dans Pocket Heist : cette formulation a servi de référence directe pour cadrer le prompt `/spec auth-state-hook`, à l'origine de la branche `claude/feature/auth-state-hook` et du fichier `_specs/auth-state-hook.md` — commit `96f5001` («docs: spec + plan pour le hook useUser (état d'auth global) », 11/08/2026, 12:51).

À quoi ressemble une spec complète — exemple useUser

Session source : 3bb6cf8f, 10/08/2026

Ce que montre la vidéo : le fichier `auth-state-management.md` du formateur, complet — Summary, Functional Requirements, Possible Edge Cases, Acceptance Criteria, Open Questions, Testing Guidelines pour le hook `useUser` (lecture seule de l'utilisateur courant, écoute temps réel via `onAuthStateChanged`, persistance après rafraîchissement, sans flux login/logout).

Ce que le formateur explique : une spec complète doit couvrir tous les cas particuliers et critères d'acceptation, même pour une fonctionnalité en apparence simple ; point pédagogique clé — les «Open Questions » ne sont pas forcément résolues par écrit, elles peuvent être discutées à l'oral pendant le mode plan (/plan), juste avant l'implémentation.

Application dans Pocket Heist : la spec initiale d'Ilies, plus courte, a été complétée pour aligner `_specs/auth-state-hook.md` sur ce niveau de détail (traduit en français) — commit `96f5001` (même commit que ci-dessus), puis implémentée en `94b159e` («feat: ajoute le hook useUser pour l'état d'authentification global », 11/08/2026, 12:55).

Du spec au plan d'implémentation détaillé — exemple Signup Firebase Integration

Session source : `3bb6cf8f`, 10/08/2026

Ce que montre la vidéo : dix captures VS Code (sous-titres français visibles, confirmant l'origine vidéo) montrant la spec `signup-firebase-integration.md` du formateur (connexion du formulaire à `createUserWithEmailAndPassword`, génération d'un «codename » aléatoire en PascalCase comme `displayName`, document Firestore `users` sans stocker l'email) puis son plan d'implémentation détaillé (fonction `generateCodename()`, gestion d'erreurs par code Firebase, tests avec mocks Vitest, vérifications finales).

Ce que le formateur explique : comment une spec se traduit en plan d'implémentation concret, étape par étape et vérifiable — pas juste une liste d'intentions.

Application dans Pocket Heist : `_specs/signup-firebase-integration.md` reprend fidèlement la génération de codename et l'exclusion de l'email du document Firestore — commit `1db9f37` («docs: spec pour Signup Firebase Integration », 11/08/2026, 13:03), implémenté ensuite en `d683af4` («feat: connecte le formulaire d'inscription à Firebase Auth », 11/08/2026, 13:10).

Applications ultérieures de la méthode (mention brève)

Plusieurs autres fonctionnalités utilisent le même couple spec/plan (bouton logout, login form, route protection, formulaire Create Heist — sessions `6b055b65` et `ecc9fb7c`) : elles appliquent une méthode déjà acquise plutôt qu'elles n'enseignent une technique nouvelle, donc pas de fiche dédiée ici. Un point mérite tout de même d'être signalé : dans `ecc9fb7c`, le premier plan proposé par Claude pour le formulaire Create Heist (avec un composant séparé `CreateHeistForm`) a été **rejeté** par Ilies puis

entièrement réécrit pour coller à la structure vue dans la vidéo du formateur — bon exemple concret de l'usage du mode plan comme étape de relecture/correction *avant* le code, implémenté ensuite dans le commit `6b9f2ec` («feat: formulaire Create Heist », 12/08/2026, 16:19).

Section 4 — MCP Servers

Cours : Claude Code Masterclass — Section 4 (13/13 leçons, 1h40).

Cette section couvre un seul serveur MCP réellement documenté par une capture de la vidéo de formation dans les 8 sessions analysées : **Firebase** (plan Firestore + Authentication). Les usages de Figma MCP et Context7 existent bien dans le projet, mais aucune image collée par Ilies ne montre le formateur en train de les expliquer — voir la note en fin de section.

Le plan Firebase MCP : Firestore + Authentication en mode test

Sessions source : 8d281e45 (10/08 matin) et 3bb6cf8f (10/08, suite — la même capture a été collée dans les deux sessions)

Ce que montre la vidéo : un terminal Claude Code appartenant au formateur (chemin visible en bas de l'image : `/Users/shaun/Code/Courses/pocket_heist`, à distinguer de l'environnement WSL d'Ilies). Le serveur Firebase MCP vient de proposer un plan en 5 étapes après la commande d'initialisation : 1. **Install Firebase SDK** — ajouter Firebase à `package.json` et créer le fichier de config SDK 2. **Initialize Firestore** — règles de sécurité en mode test (accès ouvert pour le développement) 3. **Deploy Firestore** — `firebase deploy --only firestore` 4. **Enable Authentication** — activer l'authentification Email/Password dans la console Firebase 5. **Implement Auth in App** — ajouter l'initialisation du SDK Auth dans le code

L'image montre ensuite l'appel d'outil MCP concret qui suit l'acceptation du plan : `firebase - Get Firebase SDK Config (MCP)`, avec le prompt de permission standard («Do you want to proceed? Yes / Yes and don't ask again / No »).

Ce que le formateur explique : à quoi ressemble un plan généré automatiquement par un serveur MCP «puissant» comme Firebase MCP — capable de créer de la vraie infrastructure cloud (base de données, authentification) — et pourquoi rester minimaliste dans la demande initiale (mode test uniquement, pas de hosting). Point important signalé par Claude en comparant avec l'environnement d'Ilies : le plugin Firebase MCP a changé de comportement par défaut depuis l'enregistrement de la vidéo (règles moins ouvertes par défaut aujourd'hui) — il fallait donc suivre explicitement la version du formateur pour rester en mode test ouvert.

Application dans Pocket Heist : Ilies a suivi ce plan à l'identique dans son propre terminal — installation du SDK Firebase (`npm install firebase`), écriture de `firestore.rules` en mode test ouvert (`allow read, write: if request.time < timestamp.date(2026, 9, 10);`), `firebase.json`, déploiement (`firebase deploy --only`

firestore), activation de l'authentification Email/Password, puis création de `lib/firebase/config.ts` — commit `083b5a6` («feat: configure Firebase (Firestore + Authentication) », 2026-08-11 10:44).

Note sur Figma MCP et Context7

Le projet utilise bien ces deux autres serveurs MCP concrètement : - **Figma MCP** — commit `f336a1f` («feat: style le bouton Create Heist selon le design Figma », 2026-08-11 01:08), et un tag `figma_component (if used)` ajouté au template de spec, commit `395e229` («feat: ajoute la référence Figma optionnelle à la commande /spec », 2026-08-11 01:08). - **Context7** — la consigne «vérifier la doc via Context7 avant d'écrire du code » a été ajoutée directement dans `CLAUDE.md`, commit `dadbd08` («docs: ajoute la consigne de vérification doc via Context7 dans CLAUDE.md », 2026-08-10 12:52).

Mais en vérifiant chaque image des 8 sessions (notamment l'image Figma de la session `8d281e45/3bb6cf8f`, qui montre en fait le fichier Figma **personnel** d'Ilies avec le bandeau «View only », pas une capture de la vidéo), aucune n'affiche le formateur en train d'expliquer Figma MCP ou Context7 à l'écran. Ces deux usages ont donc été appliqués sans capture vidéo correspondante dans le lot d'images fourni — pas de «leçon » sourcée pour eux, pour ne pas inventer un contenu qui n'a pas été vérifié.

Section 5 — Plugins & Skills

Cours : Claude Code Masterclass — Section 5 (6/7 leçons, 46 min).

Cette section du cours n'a laissé aucune trace directe dans les 8 sessions analysées : aucune des images collées par Ilies ne montre le formateur en train d'expliquer la notion de «Skill » ou de «Plugin » Claude Code (créer un fichier `SKILL.md`, installer un plugin, parcourir un marketplace, etc.).

Aucune leçon vidéo identifiée pour cette section

Après lecture des 8 rapports et vérification visuelle des images candidates, rien ne correspond à la Section 5 telle que décrite dans la table des matières Udemy.

Ce qui a été vérifié et écarté :

- **Le skill** `.claude/skills/firestore-schemas/Skill.md` — créé par Claude lui-même dans le commit `241a4cb` («update heist type », 12/08 12:26) pendant la génération des types Firestore, puis étendu dans `6b9f2ec` («feat: formulaire Create Heist », 12/08 16:19) après la découverte d'un bug de typage (`FirestoreDataConverter` attendait `WithFieldValue<X>` et non `Partial<X>`), documenté a posteriori pour éviter que le bug se reproduise. C'est une bonne pratique appliquée par Claude, mais **aucune image collée par Ilies ne montre le formateur créer ou expliquer ce mécanisme de skill** — l'image de la session `62cf38a7` (et sa quasi-copie dans `ecc9fb7c`) montre uniquement le texte d'un prompt sur les champs du document Firestore `heist`, rien sur la notion de skill elle-même. Cette leçon appartient plutôt à la [Section 4 \(MCP Servers / Firebase\)](#) ou reste une pratique interne à Claude, pas un contenu de la Section 5.
- **Les commandes** `/component` et `/commit-message` — présentes dans `.claude/commands/` (créées le 06/08), mais le récapitulatif de la session `8d281e45` les classe lui-même, via le sommaire réel du cours affiché à l'écran, sous «Section 2 : Commands, Context, Tools & Hooks » — pas sous Plugins & Skills. Ce sont des «commandes personnalisées » (custom slash commands), un mécanisme distinct des «skills ». Voir [section-2-commands-context-tools-hooks.md](#).
- **La commande** `/spec` (commit `d4c3bc8`) — même chose : aucune image des 8 sessions ne montre sa création depuis la vidéo ; les seules leçons vidéo capturées à son sujet portent sur le *contenu* des specs (template, sections, formulation), qui relève de la [Section 3 \(Plan Mode & Specs\)](#), déjà couverte ailleurs.
- **Les hooks** `PostToolUse` (sessions `bc2538f9` et `a056c64e`) — bien documentés, mais explicitement Section 2, pas Plugins & Skills.

Conclusion

Aucune leçon confirmée pour cette section. Les seuls objets du projet qui portent le nom «skill » (`.claude/skills/firestore-schemas/` , `.claude/skills/humanizer/`) ou qui ressemblent à des plugins ne sont pas rattachables à une capture vidéo — soit ce sont des initiatives autonomes de Claude, soit ils appartiennent aux Sections 2/3/4 déjà traitées ailleurs dans ce dossier. Si le formateur a bien consacré 46 minutes à ce sujet dans le cours, Ilies ne l'a pas encore suivi (ou n'en a pas collé de capture) au moment des 8 sessions couvertes ici.